

PLAN TÉCNICO DE DESPLIEGUE

Slotify en Producción

Guía paso a paso para desplegar la aplicación en Railway + Vercel con automatización CI/CD desde GitHub. Diseñado para alguien que parte de cero en DevOps.

€5–9	7 fases	3 ficheros	~3h
COSTE MENSUAL	PLAN DE EJECUCIÓN	CAMBIOS DE CÓDIGO	TIEMPO ESTIMADO

Proyecto Slotify - Julio 2026
Stack: NestJS + PostgreSQL + React/Vite

¿Por qué este plan?

COSTE TOTAL ~€5–9/mes Railway + Vercel gratuito	PLATAFORMA API Railway NestJS + PostgreSQL	PLATAFORMA FRONTEND Vercel React SPA — gratis
DOMINIO NUEVO ~€12/año Cloudflare Registrar		

Arquitectura objetivo

```

GitHub (rama: master) | | push / PR | | ▼ ▼
GitHub Actions CI Railway + Vercel • pnpm install Auto-deploy solo si CI ✅ • pnpm
typecheck • pnpm lint | | • pnpm test | Railway | |
NestJS API → api.slotify.es | | PostgreSQL (gestionado) | | ✅ tests OK
| | | |
| | | Vercel | |
React SPA → www.slotify.es | | WordPress sigue en
loading.es — sin ningún cambio.

```

PIEZA	PLATAFORMA	URL PÚBLICA	COSTE
API NestJS	Railway	https://api.slotify.es	~\$5-10/mes
PostgreSQL	Railway Plugin	Acceso privado + DBeaver	Incluido en Railway
Frontend React	Vercel	https://www.slotify.es	Gratis
WordPress	Loading.es	Dominio actual	Sin cambios

ANTES DE EMPEZAR

Tres conceptos que necesitas entender

1. Los dos entornos: local y producción



Entorno local (tu PC)

PostgreSQL corre en Docker (docker-compose). La API y el frontend arrancan con `pnpm dev`. Las variables de entorno están en el fichero `.env`. Solo tú puedes acceder. Datos de prueba.



Producción (internet)

PostgreSQL gestionado por Railway. La API y el frontend despliegan automáticamente. Las variables están en el panel de Railway/Vercel. Datos reales. Accessible desde cualquier lugar.



Tu `docker-compose.yml` se queda exactamente igual. Sigue siendo perfecto para desarrollo local. En producción, Railway tiene su propio PostgreSQL gestionado — los dos son independientes.

2. Los ficheros `.env` y la seguridad



Regla de oro: un fichero `.env` NUNCA debe estar en GitHub. Contiene contraseñas y claves privadas. El `.gitignore` ya lo excluye. No toques esa línea.



Local: fichero `.env`

Solo en tu disco. Nadie más lo ve. `DATABASE_URL` apunta a `localhost:5432` (Docker). Lo editas tú directamente.



Producción: panel de Railway

No hay ningún fichero `.env` en el servidor. Las variables se configuran en el panel web de

Railway/Vercel. Las guarda cifradas y las inyecta al arrancar la app.

3. Las dos bases de datos



BD local (Docker)

`localhost:5432` · Base de datos: `slotify`
Solo datos de prueba. Corre con `docker-compose up`. No importa si la rompes — es temporal.



BD producción (Railway)

Servidor remoto de Railway. Datos reales. Backups automáticos. Accesible con DBeaver usando la URL pública que da Railway. Trata sus datos con cuidado.



Cómo conectar DBeaver a la BD de Railway: Railway → tu proyecto → servicio PostgreSQL → "Connect" → copiar la "Public URL" → nueva conexión en DBeaver tipo PostgreSQL → pegar URL → activar SSL en la pestaña SSL. Listo.

PREPARACIÓN

Cuentas que necesitas crear

Antes de empezar las fases, necesitas tener estas cuentas. Todas son gratuitas.

0

Prerrequisitos

Registrar cuentas en las plataformas que usaremos — tiempo estimado: 15 minutos

1

Cloudflare (cloudflare.com)

Para registrar el dominio y gestionar DNS. Cuenta gratuita. Es el registrador más barato sin comisiones ocultas.

2

Railway (railway.app)

Para alojar el backend NestJS y la base de datos PostgreSQL. Cuenta gratuita — conéctala con tu GitHub para que tenga acceso al repositorio.

3

Vercel (vercel.com)

Para alojar el frontend React. Cuenta gratuita — también conéctala con tu GitHub. El tier gratuito de Vercel es permanente para proyectos personales.

1

Registrar el dominio

Comprar el dominio dedicado para Slotify — tiempo estimado: 10 minutos + propagación DNS

1

Buscar disponibilidad

En

cloudflare.com/products/registrar

, busca el dominio que quieres. Sugerencias ordenadas por precio: `slotify.es` (~€7/año), `slotifyapp.com` (~€10/año), `getslotify.com` (~€10/año).

2

Registrar y pagar

El proceso es inmediato. Cloudflare gestiona los DNS automáticamente — no necesitas configurar nada todavía.



¿Por qué Cloudflare? Es el único registrador que cobra exactamente el precio mayorista sin añadir margen. No tiene "precio de primer año barato + renovación cara". Y su gestor de DNS es el más rápido del mundo para propagar cambios.

2

Configurar Railway — API + Base de Datos

Desplegar el backend NestJS y provisionar PostgreSQL — tiempo estimado: 30–40 minutos

1

Crear proyecto desde GitHub

Railway → "New Project" → "Deploy from GitHub repo" → seleccionar el repositorio de Slotify. Railway detecta automáticamente que es un proyecto Node.js/pnpm.

2

Añadir PostgreSQL al proyecto

En el proyecto → "+ New" → "Database" → "Add PostgreSQL". Railway crea la base de datos y añade automáticamente la variable `DATABASE_URL` al servicio API. No tienes que configurar nada de la conexión.

3

Configurar el servicio API

En el servicio → Settings, configura:

CAMPO	VALOR
Root directory	<code>/</code> (raíz del monorepo)
Build command	<code>pnpm install --frozen-lockfile && pnpm turbo build --filter=api</code>
Start command	<code>pnpm --filter api run start:prod</code>
Watch paths	<code>apps/api/**</code>



El campo "Watch paths" hace que Railway solo redespliegue el backend cuando cambian ficheros de `apps/api/`. Sin esto, cualquier cambio en el frontend también dispararía un deploy del backend innecesariamente.

4

Configurar variables de entorno

En el servicio → pestaña "Variables". Añadir todas estas variables. Para generar los secrets, ejecuta en tu terminal (PowerShell):

PowerShell — generar secrets seguros

```
# Genera un JWT_ACCESS_SECRET seguro:
-join ((1..32) | % { '{0:X2}' -f (Get-Random -Max 256) })

# Ejecuta dos veces para tener ACCESS_SECRET y REFRESH_SECRET diferentes
```

VARIABLE	VALOR	CÓMO OBTENERLO
NODE_ENV	production	Literal
JWT_ACCESS_SECRET	cadena larga aleatoria	Comando anterior
JWT_REFRESH_SECRET	cadena larga diferente	Comando anterior (otra vez)
JWT_ACCESS_EXPIRATION	15m	Literal
JWT_REFRESH_EXPIRATION	7d	Literal
RESEND_API_KEY	tu clave	Panel de resend.com
EMAIL_FROM	noreply@slotify.es	Literal (tras activar dominio)
STORAGE_BUCKET_URL	URL de Supabase	Panel de supabase.com
STORAGE_SERVICE_KEY	clave de servicio	Panel de supabase.com
WEB_URL	https://www.slotify.es	Literal
CRON_TOKEN	cadena aleatoria corta	Mismo comando, más corto
DATABASE_URL	automático	Railway lo añade solo
PORT	automático	Railway lo añade solo

5 Configurar dominio personalizado

Settings → Networking → "Custom Domain" → introducir `api.slotify.es`. Railway mostrará un valor CNAME (algo como `tu-proyecto.up.railway.app`).

Apunta ese valor — lo necesitarás en la Fase 5.

Configurar Vercel — Frontend React

Desplegar la SPA estática con CDN global — tiempo estimado: 15 minutos

1 Importar el proyecto

Vercel → "Add New Project" → importar el repositorio de Slotify desde GitHub.

2 Configurar el build

CAMPO	VALOR
Framework Preset	Vite
Root directory	<code>apps/web</code>
Build command	<code>pnpm build</code>
Output directory	<code>dist</code>
Install command	<code>cd ../../ && pnpm install --frozen-lockfile</code>

3 Variable de entorno del frontend

Settings → Environment Variables → añadir:

VARIABLE	VALOR
<code>VITE_API_URL</code>	<code>https://api.slotify.es</code>

4 Dominio personalizado

Settings → Domains → añadir `slotify.es` y `www.slotify.es`. Vercel mostrará los registros DNS que necesitas configurar.

Apunta esos valores — los necesitarás en la Fase 5.

Vercel gestiona los certificados SSL automáticamente.

4

Cambios de código (3 ficheros)

Los únicos cambios necesarios para que la app funcione en producción — tiempo estimado: 20 minutos

Cambio 1 — `apps/api/src/main.ts`

Railway asigna el puerto dinámicamente mediante la variable de entorno `PORT`. La app actual usa `API_PORT`. Hay que hacer que acepte ambas:

```
apps/api/src/main.ts

// ❌ Antes - no funciona en Railway:
const puerto = config.get<number>('API_PORT') ?? 3000;

// ✅ Después - compatible con Railway y con local:
const puerto = parseInt(process.env.PORT ?? '') || config.get<number>('API_PORT') ?? 3000;
```

Cambio 2 — `apps/api/package.json`

Añadir el script `start:prod` que ejecuta las migraciones de Prisma antes de arrancar el servidor. Así, cada deploy aplica automáticamente los cambios de esquema a la BD de producción:

```
apps/api/package.json

{
  "scripts": {
    "build": "tsc -p tsconfig.build.json",
    "start": "node dist/main.js",
    "start:prod": "prisma migrate deploy && node dist/main.js", ← añadir esta línea
    "dev": "ts-node-dev --respawn ..."
  }
}
```

Cambio 3 — `.github/workflows/ci.yml` (fichero nuevo)

Este fichero define el pipeline de CI. GitHub lo ejecuta automáticamente en cada push y PR. Levanta una BD PostgreSQL temporal, corre los tests, y si todo pasa Railway y Vercel despliegan:

```
.github/workflows/ci.yml
```

```

name: CI

on:
  push:
    branches: [master]
  pull_request:
    branches: [master]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_USER: slotify
          POSTGRES_PASSWORD: slotify
          POSTGRES_DB: slotify_test
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
        with: { version: 11 }
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: pnpm }

      - run: pnpm install --frozen-lockfile
      - run: pnpm typecheck
      - run: pnpm lint
      - run: pnpm test
      env:
        DATABASE_URL: postgresql://slotify:slotify@localhost:5432/slotify_test
        JWT_ACCESS_SECRET: ci-test-secret
        JWT_REFRESH_SECRET: ci-test-refresh

```

```
NODE_ENV:      test
WEB_URL:       http://localhost:5173
```

5

Configurar DNS

Conectar el dominio con Railway y Vercel — tiempo estimado: 10 minutos + propagación

En el panel DNS de Cloudflare (o donde hayas registrado el dominio), añade estos registros con los valores exactos que te dieron Railway (Fase 2) y Vercel (Fase 3):

TIPO	NOMBRE	CONTENIDO	PROXY CLOUDFLARE
A	@ (apex)	IP que da Vercel para el apex	✓ Activo
CNAME	www	cname.vercel-dns.com	✓ Activo
CNAME	api	tu-proyecto.up.railway.app	✗ Solo DNS



El registro `api` NO debe tener el proxy de Cloudflare activado (la nube naranja).

Railway gestiona su propio TLS. Si activas el proxy de Cloudflare para `api`, interfiere con la negociación SSL y la conexión falla. Mantenlo en "DNS only" (nube gris).



Propagación DNS: Los cambios tardan entre 5 minutos y 2 horas en ser visibles. Puedes comprobar el estado con `nslookup api.slotify.es` en tu terminal. Vercel y Railway también muestran el estado de validación del dominio en sus paneles.

Activar CI/CD y protección de ramas

Automatizar tests y proteger la rama master — tiempo estimado: 15 minutos

1 Crear el fichero de CI

Crea la carpeta `.github/workflows/` en el proyecto y dentro el fichero `ci.yml` con el contenido del Cambio 3 (Fase 4). Haz commit y push a `master`.

2 Verificar que el pipeline pasa

GitHub → pestaña "Actions" → verificar que el workflow "CI" aparece y todos los pasos pasan en verde. El primer run puede tardar 3-5 minutos.

3 Proteger la rama master en GitHub

GitHub → Settings → Branches → "Add branch protection rule":

- Branch name pattern: `master`
- ☒ "Require status checks to pass before merging"
- → Seleccionar el check `test` (el del workflow CI)
- ☒ "Require branches to be up to date before merging"

Resultado:

nadie puede mergear código roto a master, ni siquiera por accidente.

4 Conectar Railway y Vercel con GitHub Actions

En Railway: Settings → Source → asegúrate de que está conectado a la rama `master` y "Auto-deploy" activado. En Vercel: por defecto ya está conectado a GitHub y despliega en cada push a `master`.

7

Verificación end-to-end

Comprobar que todo funciona correctamente antes de dar el despliegue por completado

1 Verificar logs de Railway

En el panel de Railway → tu servicio API → "Logs". Busca que el inicio muestre las migraciones de Prisma aplicadas y el mensaje de NestJS `Application is listening on port...`.

2 API health check

```
Terminal

$ curl https://api.slotify.es/api
# Debe responder JSON (un 404 está bien – confirma que la app está viva)

$ # Abrir en navegador:
https://api.slotify.es/api/docs
# Debe mostrar Swagger UI con la documentación de la API
```

3 Frontend accesible

Abrir `https://www.slotify.es` en el navegador. Debe cargar la SPA sin errores en la consola (F12 → Console).

4 Login end-to-end

Intentar hacer login desde la URL de producción. Esto verifica de una sola vez: que CORS está correcto entre frontend y API, que JWT funciona, y que la BD está conectada.

5 Verificar BD de producción con DBeaver

Conectarse a la BD de Railway (ver sección de conceptos) y confirmar que todas las tablas de Prisma se han creado correctamente. La tabla `_prisma_migrations` debe mostrar todas las migraciones aplicadas.

OPERACIÓN

Flujo de trabajo diario (una vez desplegado)

Tu rutina de desarrollo no cambia. El único añadido es que cada push a `master` dispara un deploy automático.



RESUMEN

Checklist de verificación

Marca cada elemento cuando lo hayas completado.

- ☐ Cuentas creadas: Cloudflare, Railway, Vercel
- ☐ Dominio registrado en Cloudflare
- ☐ Railway: proyecto creado desde GitHub
- ☐ Railway: PostgreSQL plugin añadido
- ☐ Railway: comandos build/start configurados
- ☐ Railway: todas las variables de entorno añadidas
- ☐ Railway: dominio `api.slotify.es` configurado (anotar CNAME)
- ☐ Vercel: proyecto importado desde GitHub
- ☐ Vercel: `VITE_API_URL` configurado
- ☐ Vercel: dominios `slotify.es` y `www.slotify.es` añadidos (anotar DNS)
- ☐ Código: cambio de puerto en `main.ts`
- ☐ Código: script `start:prod` en `apps/api/package.json`
- ☐ Código: fichero `.github/workflows/ci.yml` creado
- ☐ DNS: registros A, CNAME www y CNAME api configurados en Cloudflare

☐ GitHub: branch protection activada en `master`

☐ Verificación: API responde en `https://api.slotify.es/api/docs`

☐ Verificación: frontend carga en `https://www.slotify.es`

☐ Verificación: login end-to-end funciona

☐ Verificación: DBeaver conectado a BD de producción

HORIZONTE

Costes actuales y escalado futuro

ELEMENTO	AHORA (DEV/TESTING)	CON CLIENTES REALES
Railway API + BD	~\$5/mes (Starter)	~\$20/mes (Pro — más RAM, auto-scaling)
Vercel Frontend	Gratis	Gratis (o \$20/mes si necesitas analytics avanzados)
Dominio	~€12/año	Sin cambio
Monitorización	—	Sentry gratis (5.000 errores/mes) + Uptime Robot gratis
Staging environment	—	+\$5/mes en Railway (copia exacta de prod con datos sintéticos)

Nota final: Este plan está diseñado para ser ejecutado de forma incremental. Si algo falla en una fase, no afecta a las anteriores. Railway y Vercel tienen documentación excelente y soporte en sus plataformas. El orden de las fases no es arbitrario: primero registras el dominio (tarda en propagarse), luego configuras las plataformas mientras el DNS se propaga, y al final verificas todo junto.